

Exploring Discrete Soft Actor Critic for Solving Discrete POMDPs

Chulabhaya Wijesundara

Northeastern University
wijesundara.c@northeastern.edu

Abstract

We explore the problem of learning to solve discrete Partially Observable Markov Decision Processes (POMDPs) problems utilizing discrete Soft Actor-Critic (SAC). We implement a history-based recurrent version of discrete SAC and experiment on four partially observable domains: POMDP Cart-Pole, HeavenHell, Shopping, and Tiger. We also begin exploration into offline reinforcement learning with POMDPs by implementing an offline variant of our recurrent algorithm and demonstrate its performance on several varying datasets from the HeavenHell domain. Lastly, we explore several key hyperparameters of discrete SAC and show their impacts. We show that our recurrent discrete SAC algorithm can solve all four of the discrete POMDP domains tested in the off-policy setting, provide preliminary results of exploring the offline POMDP problem, and show that tuning of target entropy and optimizer epsilon can provide significant performance gains in discrete domains.

Introduction

Over the last decade, the revolution of deep learning has enabled traditional reinforcement learning algorithms to be applied to increasingly challenging and interesting problems. However, much of the work done on these problems has not been able to be transferred from the lab to real-world applications.

Among others, one of the primary reasons for this is due to the fact that most work focuses exclusively on domains that are Markov Decision Processes (MDPs), meaning that a reinforcement learning agent is able to fully observe its environment and know the complete state that it is in to select the optimal action. In the real world however, we do not have perfect information. Often times agents must deal with noisy or limited sensors preventing them from having full information about the state they are in and introducing uncertainty. These problems are known as Partially Observable Markov Decision Processes (POMDPs) and they introduce significant challenges to learning.

Several recent works have explored the application of state-of-the-art deep reinforcement learning algorithms to solving POMDPs, but these works focus primarily on continuous domains. While continuous domains do present a

challenge, there are still many real-life domains in which discrete actions are present, and solving challenging discrete POMDPs consistently still requires additional research.

An additional challenge that often prevents reinforcement learning algorithms from being applied in real-world settings is due to their live, iterative training nature. While simulator settings may allow for an agent to experience infinite sequences of trial and error to learn from its mistakes, attempting to do the same in real-world settings using physical hardware may prove catastrophic. Offline reinforcement learning seeks to solve this problem through the development of reinforcement learning algorithms that can learn performant policies solely off of static datasets of trajectories. However, much like work in traditional reinforcement learning, most work in offline reinforcement learning focuses on MDPs and has not explored the challenge of utilizing offline datasets from POMDP domains.

Motivated by these problems, in this work we explore implementing a discrete version of the state-of-the-art Soft Actor-Critic algorithm and improving it to be able to solve several discrete POMDP tasks through the use of observation history and a recurrent architecture. We show that our algorithm is able to successfully solve all of the discrete POMDP environments we experiment on, learning to utilize its memory to remember information key to solving the environments, while the standard discrete SAC algorithm fails to solve any of them. We then continue our exploration of recurrent discrete SAC into the offline reinforcement learning domain and experiment on several custom-generated datasets from the HeavenHell domain. In these mixed dataset experiments, we find a surprising result that offline recurrent discrete SAC appears to perform the best on the datasets containing the most random trajectories, as opposed to the expert dataset. Lastly, we explore several key hyperparameters to the discrete SAC algorithm and show how their tuning can dramatically impact performance.

In short, our contributions in this work can be summarized as follows:

- Develop a recurrent discrete SAC algorithm that is able to solve multiple discrete POMDP domains successfully utilizing observation history
- Develop a preliminary offline recurrent discrete SAC algorithm and experiment with custom-generated datasets from the HeavenHell domain

- Explore the impact of the target entropy scaling and Adam optimizer epsilon hyperparameters on learning performance

Background

Markov Decision Processes The traditional settings used for reinforcement learning are *Markov Decision Processes* (MDPs), which are classical formalizations of sequential decision making where actions taken by a decision-making agent can impact not only immediate, but also future rewards (Sutton and Barto 2018).

MDPs deal with the problem of learning from interactions to achieve a goal, where the problem is framed through the lens of a series of interactions between an agent and an environment. They can be represented by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, H, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{T}$ is the transition function $\mathcal{T}(s_{t+1}|s_t, a_t)$, $\mathcal{R}$ is the reward function $\mathcal{R}(s_t, a_t, s_{t+1})$, H is the horizon, and γ is the discount factor. More specifically, in a finite horizon case, at each time step over a set of discrete time steps $t = 0, 1, 2, 3, \dots, H - 1$, the agent receives a state s_t from the environment and based on that state selects an action a_t . Upon attempting this action, the environment returns a reward r_{t+1} based on the reward function and transitions to a next state s_{t+1} based on the transition function. The objective of the agent is to learn to take the best actions (a policy, π) to maximize the expected sum of discounted rewards $\mathbb{E}_\pi[\sum_{t=0}^H \gamma^t r_t]$.

Partially Observable Markov Decision Processes While MDPs are typically used to formulate many reinforcement learning problems, there is a fundamental problem with them: they are generally unrealistic. MDPs assume the *Markov property*, which means the current state always contains all the information necessary for the agent to choose the next best action. In the real-world however, an agent may not have access to perfect state information due to a variety of issues, such as noise, limited field of view, loss of communication, etc. These more complex scenarios can be modelled as *Partially Observable Markov Decision Processes* (POMDPs). POMDPs can be represented as the following tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, H, \gamma)$, which is very similar to the MDP tuple representation, except for the added Ω for the observation space, and observation function $\mathcal{O}(o_t|s_{t+1}, a_t)$. Instead of receiving true states from the environment as with an MDP, the agent instead receives an observation generated by the observation function which represents a partial state with hidden information. Therefore to gain knowledge value of the true states, the agent must use complete histories of observations.

Actor-Critic-based Deep Reinforcement Learning Although many types of algorithms exist for performing reinforcement learning with MDPs and POMDPs, the type of algorithm utilized in our work is actor-critic. Actor-critic methods concurrently learn both an *actor* to predict optimal actions given states as well a *critic* that learns a state-value function and uses it to assess the quality of actions predicted by the actor (Sutton and Barto 2018).

In order to perform learning in complex domains with actor-critic algorithms, often times non-linear function approximators in the form of artificial neural networks must be used to represent the actor and the critic. The learning of these actor and critic neural networks toward convergence is done through gradient descent; this technique is actor-critic-based deep reinforcement learning.

Soft Actor-Critic The explosion of deep learning within reinforcement learning has resulted in many variants of deep actor-critic, but the specific algorithm we utilize is known as *Soft Actor-Critic* (SAC). Unlike regular actor-critic, which seeks to learn a policy to maximize only the aforementioned expected sum of discounted rewards, SAC also attempts to maximize the entropy of the policy in order to promote better exploration. This results in the following maximum entropy objective $\mathbb{E}_\pi[\sum_{t=0}^H \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t)))]$, where α determines how much weight to give the entropy term over the traditional reward (i.e. controlling the level of exploration) (Haarnoja et al. 2018). SAC also proposes soft, entropy-augmented versions of traditional value functions and policy updates for learning.

Discrete Soft Actor-Critic The original SAC algorithm was implemented solely for use with domains with continuous action spaces which made it difficult to apply to many of the domains that exist with discrete action spaces. However SAC can be modified to work with discrete action spaces while maintaining the fundamental entropy objective (Christodoulou 2019). The key modification of discrete SAC is that it modifies the policy network to output the full action probability distribution rather a density, and then uses this distribution to directly calculate the expected value functions rather than doing Monte Carlo sampling as is necessary with continuous SAC. Mathematically, this means the soft value function changes from:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log(\pi(a_t|s_t))] \quad (1)$$

to:

$$V(s_t) = \pi(s_t)^T [Q(s_t) - \alpha \log(\pi(s_t))] \quad (2)$$

These changes were also proposed to reduce the variance with discrete SAC compared to continuous SAC (Christodoulou 2019).

Related Work

Solving Continuous POMDPs Multiple recent works have focused on exploring solving continuous POMDPs. LSTM-TD3 (Meng, Gorbet, and Kulic 2021) utilizes a similar recurrent architecture to ours, but focuses on only five continuous domains (HalfCheetah, Ant, Walker2D, Hopper, and InvertedDoublePendulum) that are made artificially partially observable. Furthermore, they utilize TD3, not SAC. Recent work focusing on baselining recurrent algorithms for POMDPs (Yang and Nguyen 2021) utilizes recurrent SAC, but again, focuses exclusively on continuous POMDP domains. Follow-up work also focusing on baselining recurrent algorithms for POMDPs (Ni, Eysenbach, and Salakhutdinov 2021) gets the closest to our work because they also implement discrete SAC and experiment on two discrete POMDP

domains, but these are pixel domains (which results in them using a different recurrent architecture) and different from ours. Most of their work also focuses on continuous domains.

Solving Discrete POMDPs Recent work on an unbiased asymmetric actor-critic methodology (Baisero and Amato 2021) focuses on solving several of the same challenging discrete POMDP domains we do, but their work utilizes a different actor-critic algorithm, A2C. Although A2C is an actor-critic method like SAC, the reason we utilize SAC for our work is that it is more similar to another actor-critic method TD3 which has also been shown to successfully work in offline reinforcement learning scenarios, but only with continuous environment datasets. Therefore utilizing SAC, which has been proven to work in discrete environments and is also state-of-the-art, would allow us to smoothly transition to exploring solving discrete POMDPs in an offline reinforcement learning setting.

Solving Image-based POMDPs DRQN (Hausknecht and Stone 2015) implements a similar recurrent architecture to ours, but utilize DQN as the fundamental reinforcement learning algorithm. Furthermore, the POMDPs they explore have image-based observations, rather than the continuous vector or discrete observations utilized for the POMDP domains we explore.

Project Description

Our work explores the problem of learning to solve several diverse discrete POMDP problems in both an off-policy capacity, and in a simple offline capacity.

Discrete Soft Actor-Critic The primary algorithm we implement for our exploration with discrete POMDP environments is discrete SAC. In taking an existing implementation of continuous SAC (Huang et al. 2022) and modifying it to work for discrete settings, we programatically implement the several algorithmic changes proposed in the discrete SAC work (Christodoulou 2019).

The first is the modification of the actor and critic networks. We modify the actor network to output a probability distribution rather than a density function. This simplification is allowed for the discrete setting due to the limited size of the action space (as opposed to infinite for continuous actions). In selecting actions, we then simply sample from the categorical distribution, rather than a re-scaled Gaussian as with continuous SAC. We modify the critic network in a similar fashion. Because there are a limited number of actions in a discrete setting, we use a DQN-style architecture that takes a state as input and outputs the Q-values for all actions. This is in contrast to continuous SAC, which outputs a single Q-value for an input of a state-action pair.

The second set of algorithmic changes we make are to the objective functions used in continuous SAC. The form of the objective for the soft Q-function remains the same as continuous SAC (based on standard one-step TD error), but we modify the value function to directly calculate the expectation using the now-available action probability distribution instead of calculating an expectation through Monte

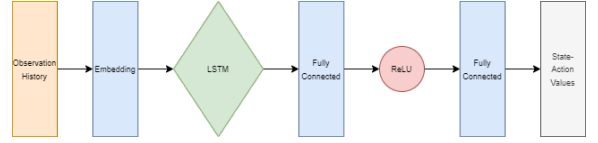


Figure 1: Recurrent critic network architecture

Carlo sampling. This results in the update from (1) to (2) described above. We apply the same direct expectation calculation versus Monte-carlo sampling to the objectives for the policy and the temperature. This changes the policy objective from (3) to (4):

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim D, \epsilon_t \sim N} [\alpha \log(\pi_{\phi}(f_{\phi}(\epsilon_t; s_t) | s_t)) - Q_{\theta}(s_t, f_{\phi}(\epsilon_t; s_t))] \quad (3)$$

(where $f_{\phi}(\epsilon_t; s_t)$) represents the reparametrization trick (Kingma et. al., 2013) to allow gradient backpropagation for learning).

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim D} [\pi_t(s_t)^T [\alpha \log(\pi_{\phi}(s_t)) - Q_{\theta}(s_t)]] \quad (4)$$

We update the temperature objective from (5) to (6) as follows:

$$J(\alpha) = \mathbb{E}_{a_t \sim \pi_t} [-\alpha(\log \pi_t(a_t | s_t) + \bar{H})] \quad (5)$$

$$J(\alpha) = \pi_t(s_t)^T [-\alpha(\log(\pi_t(s_t)) + \bar{H})] \quad (6)$$

Recurrent Discrete Soft Actor-Critic The base discrete SAC algorithm assumes the Markov property; the actor and critic networks take in only a single timestep of fully observable state as input to predict actions and values. However, POMDPs violate the Markov property due to the environment returning partially observable observations rather than fully observable states. We modify the discrete SAC algorithm to account for this partial observability challenge by taking into account a *flexible length history*, a sequence of past observations and actions.

The default network architecture for discrete SAC is entirely fully connected, and thus cannot learn time-based dependencies due to lack of memory. We keep the fully connected architecture, but add a single intermediate recurrent Long Short-Term Memory (LSTM) layer (Hochreiter and Schmidhuber 1997) that resolves this issue and allows the learning process to learn long-term dependencies using the history of observations (the critic architecture is shown in Figure 1). We also further modify the architecture to output predictions for actions and values at all timesteps of the history, rather than a single timestep prediction. We further modify the calculation of all of the objective functions described earlier to account for history h_t , where $h_t = o_0, o_1, o_2, \dots, o_{t-1}$ is an array containing a sequence of observations o from an episode sampled from a replay buffer at a specific time t .

$$V(h_t) = \pi(h_t)^T [Q(h_t) - \alpha \log(\pi(h_t))] \quad (7)$$

$$J_{\pi}(\phi) = \mathbb{E}_{h_t \sim D} [\pi_t(h_t)^T [\alpha \log(\pi_{\phi}(h_t)) - Q_{\theta}(h_t)]] \quad (8)$$

$$J(\alpha) = \pi_t(h_t)^T [-\alpha(\log(\pi_t(h_t)) + \bar{H})] \quad (9)$$

History-based Replay Buffer Due to being an off-policy algorithm, SAC utilizes a replay buffer to store and sample transitions from for learning. However, this standard replay buffer can only sample batches of single timesteps, which makes it impossible to use with the recurrent discrete SAC algorithm described in the prior section. Therefore we implement a custom replay buffer that is able to sample from the full spectrum of single timestep batches to full episode batches (with intermediate fixed history length batches in between). Variable length episodes are batched together by padding all episodes to the length of the longest episode. We ensure that this padding does not affect gradient calculations by calculating a mask variable that tracks which entries of the history array are padding, and normalize the objective calculations by the sum of the non-zero entries of this mask (i.e. normalizing by only the amount of non-padding entries in the history array).

Offline Reinforcement Learning The last several years has seen a surge of research into offline reinforcement learning. Unlike online or off-policy reinforcement learning, offline reinforcement learning provides a unique challenge by completely removing the ability for the agent to interact with the environment and receive iterative feedback; there is only a fixed dataset from which it must learn the best possible policy. The chief challenge that this presents is that of distribution shift, where the agent’s learned values drift from the dataset over time due to accumulated error and having no ability to interact with the environment and receive feedback for correcting that accumulated error.

To our knowledge, there is currently no experimental work investigating solving any POMDPs, continuous or discrete, in the offline reinforcement learning setting. Therefore we begin exploration down this path. In particular we implement a completely offline version of discrete SAC where all environment interaction is removed. Furthermore, we create scripted policies for a discrete POMDP environment, generate datasets of varying levels of expert trajectories, and attempt preliminary training.

Experiments

Our work experiments with several different POMDP domains that provide a variety of different challenges, where solving these POMDP domains completely should generally be impossible for standard discrete SAC. We further perform preliminary experiments into offline reinforcement learning with one of these domains to explore how our algorithm can handle offline data. Lastly, we provide some preliminary results of investigating two specific hyperparameters of interest, the target entropy value and the epsilon of the Adam optimizer to show two areas where performance of our algorithm can likely be easily improved before resorting to additional hyperparameter tuning.

Domains We investigate the solving of four different discrete POMDP problems. We utilize the classic CartPole domain converted into a POMDP problem: CartPole-P, where the velocity observations are removed and the agent must learn to remember the position observations to derive them.

We also investigate several more challenging POMDPs that test the memorization capabilities of the agent: HeavenHell, Shopping, and Tiger. HeavenHell requires an agent to navigate to a priest which informs the agent of the location of heaven, remember the location, and then correctly navigate to that location. Going to the correct location gives a reward of +1 and the incorrect location a penalty of -1.

Shopping requires an agent in a small gridworld to buy an item at a specific location on the grid. The agent may query the location of the item at a small penalty of -2, and move at a small penalty of -1. Buying the incorrect item gives a larger penalty of -5, and buying the correct item gives a reward of +10. Therefore an agent must learn to query as little as possible and remember the location of the item they must buy.

Lastly, we experiment on the Tiger domain which introduces challenging stochasticity. In Tiger, an agent must open one of two doors, one of which may contain a tiger behind it. If they open the door with the tiger behind it, the agent receives a large penalty of -100. If they open the door without a tiger, they receive a smaller reward of +10. The agent also has the option to take the action to listen at a small penalty, which informs the agent which door to open. However, there is still a 15% chance this door may have a tiger behind it. Therefore, the agent must not only learn to listen to have the best chance at opening the right door, but also learn the statistics that tell it that listening and opening will be on average better than simply guessing.

Experimental Setup

All experiments were run on a Dell Precision 7540. Due to lack of proper GPU compute and thus experiments taking too long to run, we limited all our experiments to 100,000 timesteps. Furthermore, we were not able to run extensive hyperparameter tuning due to this setup and kept all of our experiments at the original SAC hyperparameters (i.e. same learning rates for actor, critic, same network update frequencies, etc.).

Results

POMDP Cartpole When training our algorithm on the POMDP Cartpole environment (Figure 2), we see that our history-based recurrent discrete SAC algorithm is able to learn how to derive position and thus consistently solve the problem (although the variance is high). Default discrete SAC on the other hand is unable to learn.

HeavenHell In the HeavenHell domain (Figure 3), we see that our history-based recurrent discrete SAC algorithm learns to consistently get to the correct heaven position by learning to remember the location given by the priest. The default discrete SAC algorithm however learns two policies during the training period, both of which have an expected return of 0. It either decides to guess which goal location is heaven or hell (thus averaging receiving +1s and -1s), or it waits until the episode times out and does not guess at all (consistently receiving returns of 0).

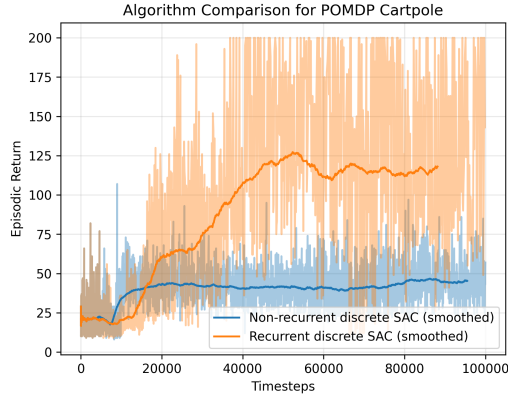


Figure 2: Recurrent vs. non-recurrent algorithms on POMDP Cartpole

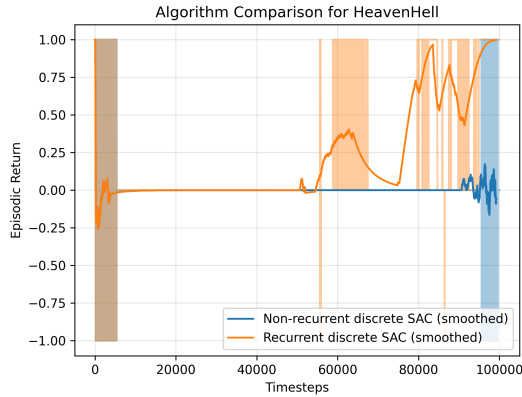


Figure 3: Recurrent vs. non-recurrent algorithms on HeavenHell

Shopping Our algorithm also shows a significant improvement over the non-recurrent discrete SAC baseline in the Shopping domain (Figure 4). The baseline algorithm learns a policy to never attempt to buy at all, as attempting to buy at the wrong location would give a significant penalty of -5. Because there is no action that allows it to remain in the same location, it must constantly move and thus times out the episode with a total return of -50 (because moving at each timestep has a penalty of -1). However, our history-based recurrent discrete SAC algorithm learns to query and remember the location of the item it needs to buy, thus enabling it to obtain significantly better return.

Tiger The results in the Tiger domain (Figure 5) are harder to discern than the prior tasks, though it is arguably also the hardest of the tested domains. The additional challenge of the Tiger environment compared to the other domains is that of stochasticity. Even when the agent takes the right action and chooses the right door, there is a significant 15% chance that they will be unlucky and lose, thus receiving a large

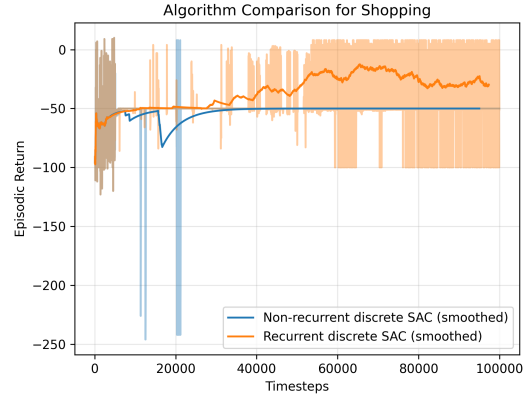


Figure 4: Recurrent vs. non-recurrent algorithms on Shopping

penalty of -100. We can still see a clear difference between the learned policies of the algorithms however. The baseline discrete SAC algorithm clearly learns only to randomly pick which door to open in a 50/50 gamble, where it either receives +10 for guessing the right door, or -100 for guessing the wrong door. The recurrent algorithm however initially learns a policy of consistently listening instead of guessing, and later into training starts learning to guess after listening. Although in this specific experiment we are unable to show definitively that our algorithm performs better, we show later that under improved hyperparameters that our recurrent algorithm does indeed learn to consistently solve this environment.

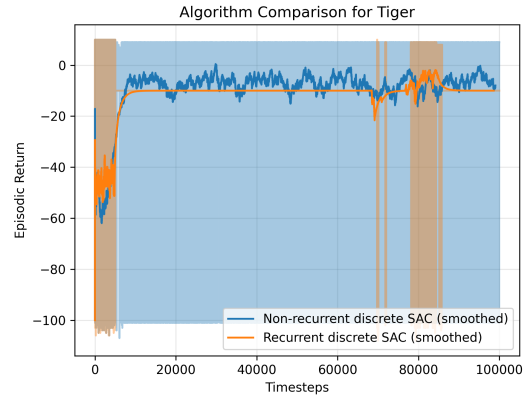


Figure 5: Recurrent vs. non-recurrent algorithms on Tiger

Offline HeavenHell In our preliminary exploration into offline reinforcement learning on POMDPs, we experiment with how much can be learned off of offline datasets of varying levels of expert data. We specifically utilize the HeavenHell domain due to the sparse rewards adding additional difficulty.

Rather surprisingly, our results (Figure 6) show that the more random the trajectories in the dataset, the better the learning performs. For instance, we see that the expert dataset learning initially learns to solve the task, but quickly fails and does not recover. However the 50% and 100% random datasets both learn to consistently solve the environment, and the 75% environment also learns to solve for a significant period of time before failing. We hypothesize that this may be related to the fundamental problem of offline reinforcement learning: distribution shift. When the agent takes an incorrect action with the expert dataset, it has no other trajectories that may match that incorrect action to use as feedback in the dataset. However the random datasets are likely to have seen that same trajectory, thus allowing for learning across varying trajectories over time.

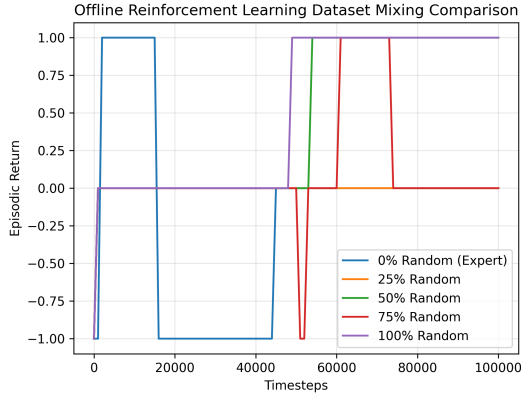


Figure 6: Offline learning comparison among varying HeavenHell datasets

Target Entropy Tuning Although we were not able to perform extensive hyperparameter tuning due to lack of compute, one specific hyperparameter we explored was the target entropy. In all of the aforementioned experiments, we utilize the same target entropy as the continuous SAC algorithm: $\bar{\mathcal{H}} = -\dim(\mathcal{A})$ (the original SAC work does not provide any explanation as to why this particular value was chosen).

However, this value is clearly sub-optimal in the discrete action setting due to the fact that entropy in discrete settings must be positive. Using continuous SAC’s target entropy value in the discrete setting results in the agent’s exploration dropping to extremely small levels very quickly due to the fact that it is trying to hit an impossible entropy value in optimization (and thus reducing the exploration coefficient α to nearly zero).

We compare implementing the target entropy recommended by Christodoulou and find that while the general target entropy value they recommend is suboptimal for our domains, the formulation provides better results when tuned. Specifically, they recommend a target entropy formulation of $\bar{\mathcal{H}} = -\delta(\log(1/\dim(\mathcal{A})))$, where δ is a scaling factor, and recommend using $\delta = 0.98$. This formulation intuitively

means that in trying to reach this target entropy value, the optimization process is attempting to drive the action selection to be from a uniform distribution, and thus promote maximum exploration.

Our results (Figure 7) show that using such a high scaling value as recommended by Christodoulou leads to too much exploration and no learning to solve the POMDP Cartpole task. In exploring varying scaling values, we see that three lower scaling factors of 0.01, 0.1, and 0.3 all lead to significantly better learning than our baseline which used the target entropy from the continuous SAC algorithm. We hypothesize that this is due to the fact that smaller action spaces need less exploration for determining optimal actions, and thus require smaller scaling factors (but not too small, as we see that 0.01 performs worse than 0.1 or 0.3).

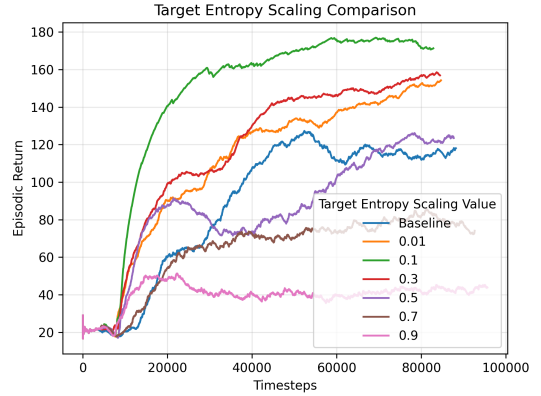


Figure 7: Comparison among different scaling values for target entropy using POMDP Cartpole

Optimizer Epsilon Tuning To verify our claim of a more optimal target entropy value for smaller action spaces holds, we test on the HeavenHell domain. Furthermore, we test changing a different hyperparameter: the epsilon of the Adam optimizer. In all our prior experiments we use the default value of epsilon suggested by the PyTorch framework of $1 * 10^{-8}$. However, while not mentioned in their work, Christodoulou utilize a different, larger value of $1 * 10^{-4}$ in their code. We experiment with this value and find that it provides significant learning improvements (Figure 8), potentially due to the fact that the larger the epsilon used for Adam, the smaller the gradient step update, thus reducing variance. We see that using both an updated target entropy with a scaling of -0.3 and an updated Adam epsilon value results in significantly faster and stabler learning of the HeavenHell domain.

Seeding Experiment In order to better verify the statistical significance of our algorithm’s improvements versus the non-recurrent discrete SAC baseline, we experimented with three additional seeds using the updated hyperparameter configuration described in the earlier sections (i.e. using an updated target entropy and Adam optimizer epsilon

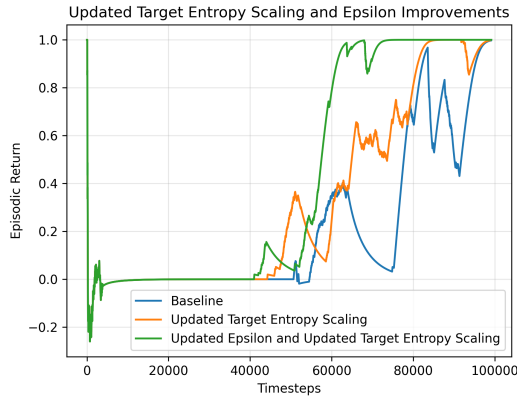


Figure 8: Comparison of performance improvements given by scaling target entropy and modifying optimizer epsilon

value). We run on the Tiger task and see that our recurrent algorithm is able to consistently learn to solve the problem across all seeds while the non-recurrent algorithm struggles. However this is still only with three seeds, and with more compute we would like to run on a significantly larger set of seeds in the future.

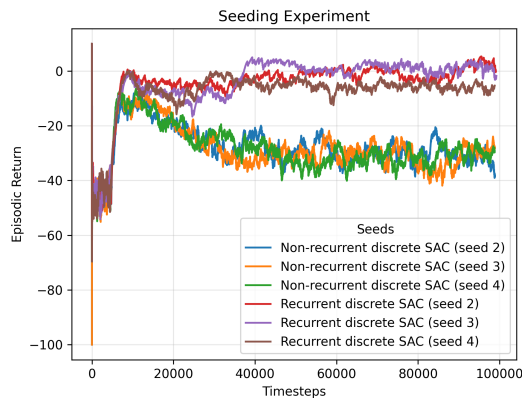


Figure 9: Comparison of recurrent vs. non-recurrent discrete SAC on Tiger with varying seeds

Conclusion

In this work we develop a recurrent discrete SAC algorithm and apply it to four different discrete POMDP domains and show that it is able to successfully solve all of them. In addition, we provide initial surprising results of our offline recurrent discrete SAC algorithm performing on several HeavenHell datasets of mixed randomness levels. Lastly, we show that tuning the target entropy scaling and Adam optimizer epsilon values can dramatically impact performance and thus should be paid attention to in future work.

In trying to complete this project we learned about the

ins and outs of the SAC algorithm, its discrete variant, and learning to implement history-based modifications to algorithms to successfully solve POMDPs. We also learned more about performing offline reinforcement learning by first-hand running into distribution shift issues while attempting to train on the HeavenHell datasets. Lastly, we learned more about the significant impacts of certain hyperparameters, particularly those involved in specifying the exploration levels of the algorithm. Our code is located here: <https://github.com/Chulabhaya/recurrent-discrete-sac>

For future works we would like to continue exploration into offline reinforcement learning research with POMDPs and potentially develop improvements to SAC based on challenges that we encounter. We would also like to obtain significantly more compute to allow for more experimentation, hyperparameter tuning, as well as validation of results through thorough seeding experiments.

References

- Baisero, A.; and Amato, C. 2021. Unbiased Asymmetric Actor-Critic for Partially Observable Reinforcement Learning. *CoRR*, abs/2105.11674.
- Christodoulou, P. 2019. Soft Actor-Critic for Discrete Action Settings. *CoRR*, abs/1910.07207.
- Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *CoRR*, abs/1801.01290.
- Hausknecht, M. J.; and Stone, P. 2015. Deep Recurrent Q-Learning for Partially Observable MDPs. *CoRR*, abs/1507.06527.
- Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Comput.*, 9(8): 1735–1780.
- Huang, S.; Dossa, R. F. J.; Ye, C.; Braga, J.; Chakraborty, D.; Mehta, K.; and Araújo, J. G. 2022. CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms. *Journal of Machine Learning Research*, 23(274): 1–18.
- Meng, L.; Gorbet, R.; and Kulic, D. 2021. Memory-based Deep Reinforcement Learning for POMDP. *CoRR*, abs/2102.12344.
- Ni, T.; Eysenbach, B.; and Salakhutdinov, R. 2021. Recurrent Model-Free RL is a Strong Baseline for Many POMDPs. *CoRR*, abs/2110.05038.
- Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. The MIT Press, second edition.
- Yang, Z.; and Nguyen, H. 2021. Recurrent Off-policy Baselines for Memory-based Continuous Control. *CoRR*, abs/2110.12628.